Aprendizaje Automático - Libro 1 - Clase 1 a 7

5. Clase 5

Clase Nº 5: Aprendizaje Supervisado (II)

Contenido: K-Nearest Neighbors (K-NN). Árboles de decisión.

En la clase de hoy trabajaremos los siguientes temas:

- ¿Cómo funciona K-NN?
- Regresión con K-NN
- Clasificación con Árboles de Decisión

1. Presentación:

Bienvenidos a la clase N° 5 de Aprendizaje Automático.

En esta clase de Aprendizaje Automático, exploraremos dos poderosos algoritmos: K-Nearest Neighbors (K-NN) y Árboles de Decisión. Comprenderemos cómo estos algoritmos son fundamentales en la construcción de modelos predictivos y de clasificación. Con K-NN, aprenderemos a identificar patrones cercanos en conjuntos de datos, lo que lo convierte en una herramienta valiosa para la clasificación y la regresión. Por otro lado, los Árboles de Decisión nos permitirán tomar decisiones lógicas basadas en características específicas, lo que es esencial para la toma de decisiones automatizadas. Este concepto nos ayudará a desarrollar soluciones de machine learning para los casos de uso que van desde la recomendación de productos y diagnóstico médico hasta la detección de fraudes y el análisis de sentimientos en redes sociales. Estos algoritmos ofrecen una base sólida para adentrarse en el emocionante mundo del Aprendizaje Automático y sus aplicaciones en la resolución de problemas del mundo real.

2. Desarrollo y Actividades:

Los conceptos que estudiaremos en esta clase se relacionan con lo que venimos viendo anteriormente en la clase N° 4.

Si no recuerdas estos temas te invitamos a que visites el campus revises nuevamente la clase N°4 o veas el video. Si te quedan dudas sobre este tema siempre podés consultar en el foro o en las tutorías cuando nos encontremos.

K-Nearest Neighbors (K-NN)

El algoritmo (k-NN) es una forma de aprendizaje automático supervisado que se utiliza para predecir categorías, *sklearn.neighbors* proporciona funcionalidad para los métodos de aprendizaje basados en vecinos supervisados y sin supervisión. El aprendizaje supervisado basado en vecinos se presenta en dos tipos:

- Clasificación para datos con etiquetas discretas
- Regresión para datos con etiquetas continuas.

El principio detrás de los métodos del vecino más cercano es encontrar un número predefinido de ejemplos de entrenamiento más cercanos a la distancia al nuevo punto, y predecir la etiqueta a partir de estos. El número de muestras puede ser una constante definida por el usuario, o variar según la densidad local de puntos. La distancia puede ser, en general, cualquier medida métrica, los métodos basados en vecinos se conocen como métodos de aprendizaje automático no generalizados, ya que simplemente "recuerdan" todos sus datos de entrenamiento.

A pesar de su simplicidad, los vecinos más cercanos han tenido éxito en una gran cantidad de problemas de clasificación y regresión, en áreas de salud, análisis de imágenes satelitales, finanzas, ciencias políticas, reconocimiento de vides y más. Al ser un método no paramétrico, a menudo es exitoso en situaciones de clasificación donde el límite de decisión es muy irregular. Podría ser aplicado con mucho éxito en ambientes políticos complejos para clasificar votantes potenciales de un determinado candidato.

KNN es un algoritmo de aprendizaje no *paramétrico* y *perezoso*.

- *No paramétrico* significa que no hay suposiciones para la distribución de datos subyacentes, por lo que es muy útil en la práctica donde la mayoría de los conjuntos de datos del mundo real no siguen supuestos teóricos matemáticos.
- *Algoritmo perezoso* significa que no necesita datos de entrenamiento para la generación del modelo. Todos los datos de entrenamiento son utilizados en la fase de prueba. Esto hace que la capacitación sea más rápida y la fase de prueba más lenta y costosa. La fase de prueba costosa significa más tiempo y memoria. En el peor de los casos, KNN necesita más tiempo para escanear todos los puntos de datos y escanear todos los puntos de datos requerirá más memoria para almacenar datos de entrenamiento.

K-NN Clasificación

La clasificación basada en vecinos es un tipo de aprendizaje basado en instancias o no generalizado, no intenta construir un modelo interno general, sino que simplemente almacena instancias de los datos de entrenamiento. La clasificación se calcula a partir de una mayoría simple de votos de los vecinos más cercanos de cada punto, a un punto de consulta se le asigna la clase de datos que tiene más representantes dentro de los vecinos más cercanos de dicho punto. *scikit-learn* implementa dos clasificadores de vecinos más cercanos:

- *KNeighborsClassifier* implementa el aprendizaje basado en los *k* vecinos más cercanos de cada punto de consulta, donde *k* es un valor entero especificado por el usuario.
- *RadiusNeighborsClassifier* implementa el aprendizaje basado en el número de vecinos dentro de un radio *r* fijo de cada punto de entrenamiento, donde *r* es un valor de punto flotante especificado por el usuario.

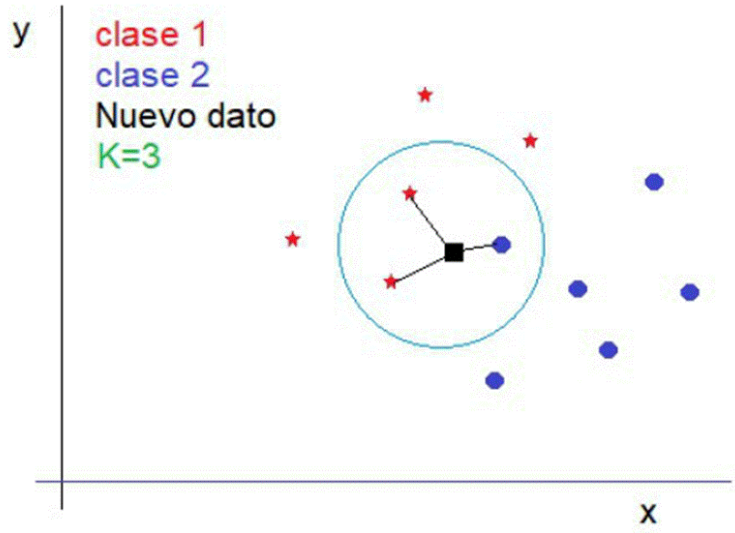
La clasificación de vecinos en *KNeighborsClassifier* es la técnica más utilizada, la elección óptima del valor *k* depende en gran medida de los datos, en general, un valor más alto de *k* suprime los efectos del ruido, pero hace que los límites de clasificación sean menos definidos. En los casos en que los datos no se muestren de manera uniforme, la clasificación de vecinos basada en el radio puede ser una mejor opción. El usuario especifica un radio fijo *r*, de modo que los puntos en vecindarios más dispersos usan menos vecinos cercanos para la clasificación. Para espacios de parámetros de alta dimensión, este método se vuelve menos efectivo debido a la llamada "*maldición de la dimensionalidad*".

La clasificación básica de vecinos más cercanos utiliza ponderaciones uniformes: es decir, el valor asignado a un punto de consulta se calcula a partir de una mayoría simple de votos de los vecinos más cercanos. En algunas circunstancias, es mejor ponderar a los vecinos de modo que los vecinos más cercanos contribuyan más al ajuste. Esto se puede lograr a través de la palabra clave *weights*:

- *weights* = '*uniform*', asigna pesos uniformes a cada vecino.
- *weights* = '*distance*' asigna pesos proporcionales al inverso de la distancia desde el punto de consulta. Alternativamente, se puede suministrar una función de la distancia definida por el usuario para calcular los pesos.

Cómo funciona

Supongamos que tenemos una muestra clasificada en dos clases (*clase 1* y *clase 2*) según se muestra a continuación, y tomemos el valor de *k = 3*.



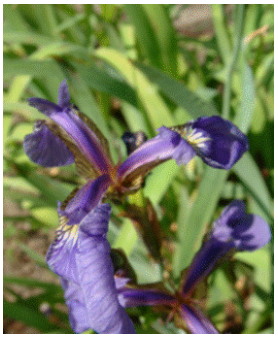
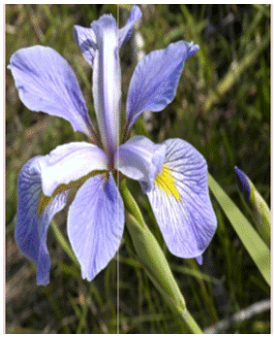
En este caso dado un nuevo punto a clasificar, se realizan los siguientes pasos:

1. Se calcula la distancia a los datos.
2. Se encuentran los tres más cercanos.
3. Se clasifica según categoría de la mayoría.

Ejemplo de clasificación mediante KNN

Veamos un ejemplo que podemos obtener de la página de *scikit-learn*, en el cual se utilizan los datos de tres variantes de la flor "Iris" para realizar una *clasificación*.

Iris Setosa	Iris Versicolor	Iris Virginica

		
Referencia autor: CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=170298	Referencia autor: CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=248095	Referencia autor: De Frank Mayfield - originally posted to Flickr as Iris virginica shrevei BLUE FLAG, CC BY-SA 2.0, https://commons.wikimedia.org/w/index.php?curid=9805580

La clasificación se podría realizar en función de:

- El largo del sépalo.
- El ancho del sépalo.
- El largo del pétalo.
- El ancho del pétalo

En esta oportunidad realizaremos la clasificación por:

- El largo del sépalo.
- El ancho del sépalo.

Los datos fueron recopilados por Edgar Anderson, los cuales fueron introducidos por Ronald Fisherd en 1936 y se conocen como *Iris conjunto de datos de Fisherd*. Este conjunto de datos se convirtió en un caso de prueba típico para muchas técnicas de clasificaciones estadísticas en aprendizaje automático.

Paso 1

Primero debemos importar las librerías a utilizar:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn import neighbors, datasets
```

Paso 2

De datasets importamos el conjunto de datos de *iris()* y establecemos el número de vecino más cercano en *15*

```
iris = datasets.load_iris()
n_neighbors = 15
```

Paso 3

La librería Iris se encuentra separada en:

- *data* que contiene todas las características.
- *target* que contiene las clases asociadas a esas características.

Los valores de *target* no se encuentran como string sino que tienen asociado para cada string un valor numérico de forma de poder analizar los datos. Como dijimos antes, del conjunto de datos posibles vamos a tomar solo las dos primeras características por lo que podemos representar los datos de la siguiente manera.

```
X = iris.data[:,:2] # solo tomaremos los primeros 2 features, para ejemplificar.
print(X)
y = iris.target
print(y)
```

En este caso en *data* estamos indicando *[filas, columnas]*, con dos puntos (:) estamos indicando todas las filas y con *:2* (o *0:2*) indicamos las primeras dos columnas. También podríamos indicar otras columnas especificando el rango como 1:3 en donde podríamos tomar de la segunda columna a la tercera.

Paso 4

Para analizar los datos visualmente crearemos un mapa de colores mediante *ListedColormap* a partir de una lista de colores, y utilizaremos un mallado con un paso de 0.2

```
h = .02
cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
cmap_bold = ListedColormap(['#FF0000', '#00FF00', '#0000FF'])
```

Paso 5

Ahora realizaremos dos clasificaciones, una para un peso uniforme y otra para un peso en función al inverso de la distancia.

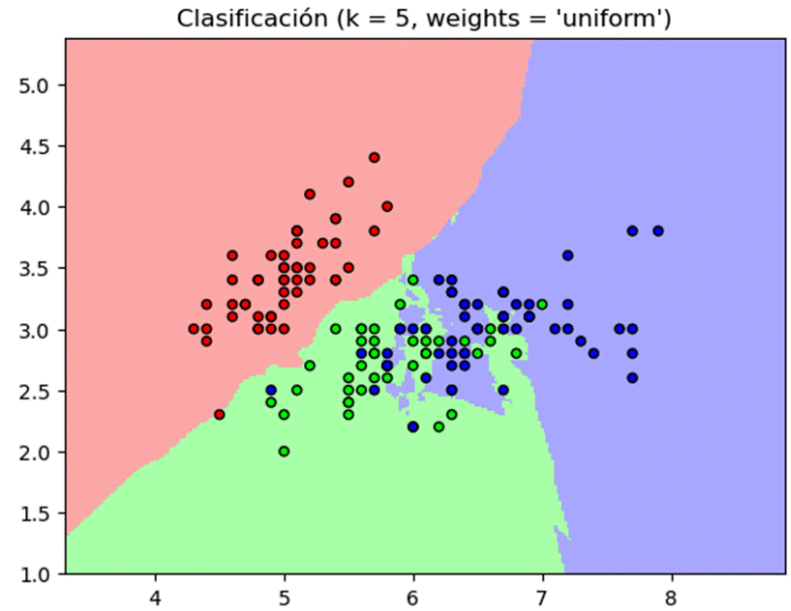
```
 n_neighbors = 5
for weights in ['uniform', 'distance']:
    # Creamos una instancia del clasificador de vecinos más cercanos y le pasamos los datos mediante fit().
    # El primer párametro de KNeighborsClassifier es con cuantos vecinos quiero clasificar y el
    # segundo el tipo de peso a utilizar.
    clf = neighbors.KNeighborsClassifier(n_neighbors, weights=weights)
    clf.fit(X, y)

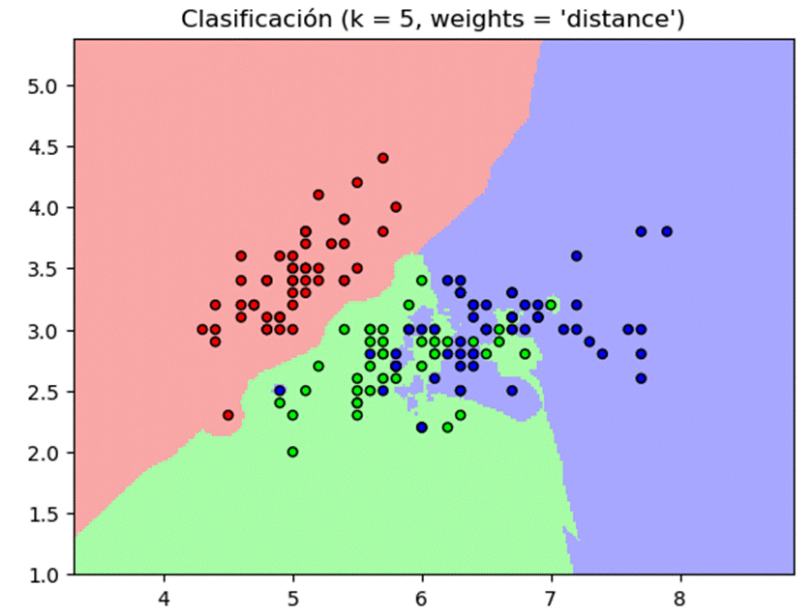
    # Establecemos los límites del gráfico y asignamos un color a cada punto de malla.
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])

    # Agregamos el resultado al gráfico
    Z = Z.reshape(xx.shape)
    plt.figure()
    plt.pcolormesh(xx, yy, Z, shading='auto', cmap=cmap_light)

    # Ploteo los datos de entrenamiento
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap_bold, edgecolor='k', s=20)
    plt.xlim(xx.min(), xx.max())
    plt.ylim(yy.min(), yy.max())
    plt.title("Clasificación (k = %i, weights = '%s')" % (n_neighbors, weights))

plt.show()
```





Si ahora creamos un conjunto de datos Xn a testear y lo incluimos dentro del bucle for podemos obtener la predicción mediante ambos pesos, según se muestra a continuación:

```
#Ploteo un nuevo dato
Xn = np.array([[7.3,3], [5.1,2.9], [6.4,3.2]])
Yn = clf.predict(Xn)
print(Yn)
```

Resultado:

weight = uniform	**weight = distance**
[2 0 2]	[2 0 1]

K-NN Regresión

La regresión basada en vecinos se puede utilizar en los casos en que las etiquetas de datos son variables continuas en lugar de discretas. La etiqueta asignada a un punto de consulta se calcula en función de la media de las etiquetas de sus vecinos más cercanos. *scikit-learn* implementa dos tipos de regresiones diferentes:

- *KNeighborsRegressor* implementa el aprendizaje basado en los vecinos más cercanos de cada punto de consulta, donde r es un valor entero especificado por el usuario.
- *RadiusNeighborsRegressor* implementa el aprendizaje basado en los vecinos dentro de un radio r fijo del punto de consulta, donde r es un valor de punto flotante especificado por el usuario.

La regresión básica de vecinos más cercanos utiliza pesos uniformes, es decir, cada punto en el vecindario local contribuye de manera uniforme a la clasificación de un punto de consulta. Bajo algunas circunstancias, puede ser ventajoso ponderar los puntos de manera tal que los puntos cercanos contribuyan más a la regresión que los puntos lejanos. Esto se puede lograr a través de la palabra clave *weights* (pesos). El valor predeterminado, *weights = 'uniform'*, asigna pesos iguales a todos los puntos mientras que *weights = 'distance'* asigna pesos proporcionales al inverso de la distancia desde el punto de consulta. Alternativamente, se puede suministrar una función de la distancia definida por el usuario, que se usará para calcular los pesos.

Árboles de decisión

Un árbol de decisiones se asemeja a las raíces de un árbol, en donde partimos de un conjunto de datos con determinadas características, que llamaremos raíz principal y que iremos descomponiendo por atributos, en ramas a partir de una determinada clasificación. Cada descomposición lleva asociada una condición que puede resultar verdadera o falsa y que se encuentra relacionada a una caracterización específica.

Podríamos tener por ejemplo el atributo *"tipo de vehículo"* con valores:

- Camionetas
- Autos

Y el atributo *"tracción"*, con valores:

- Cuatro ruedas
- Dos ruedas

En base a estos atributos podríamos crear un árbol en el cual la primera división se realice por *"tipo de vehículo"* y luego por *"tracción"* o al revés. Esta división la realizaremos a partir de un algoritmo que optimice la forma en la cual se lleva a cabo la división en base a un análisis probabilístico.

Cuanto más profundo es el árbol, más complejas son las reglas de decisión y más se ajusta el modelo

Algunas ventajas de los árboles de decisión son:

- Sencillos de entender e interpretar.
- Los árboles se pueden visualizar.
- Requiere poca preparación de datos.
- Permite trabajar con datos tanto numéricos como categóricos.
- Permite manejar problemas de salida múltiple.
- Posible validar un modelo utilizando pruebas estadísticas. Eso permite dar cuenta de la fiabilidad del modelo.

Algunas desventajas de los árboles de decisión incluyen:

- Los aprendices de árboles de decisión pueden crear árboles demasiado complejos que no generalizan bien los datos. Esto se llama sobreajuste.
- Los árboles de decisión pueden ser inestables porque las pequeñas variaciones en los datos pueden dar lugar a que se genere un árbol completamente diferente.
- Los aprendices de árboles de decisión crean árboles sesgados si algunas clases dominan. Por lo tanto, se recomienda equilibrar el conjunto de datos antes de ajustar con el árbol de decisión.

Instalaciones adicionales opcionales

Para poder trabajar cómodamente con árboles de decisiones, podemos instalar un par de librerías, que nos van a ayudar a visualizar los datos, para ello seguiremos los siguientes pasos:

Instalamos Graphviz

- Vamos a la página de graphviz y descargamos el ejecutable para nuestro sistema operativo: https://graphviz.org/download/
- Una vez descargado simplemente damos siguiente hasta terminar la instalación y nos fijamos en donde es que se va a instalar. En el caso de que este trabajando en Windows, es posible que, se instale en *C:\Program Files (x86)\Graphviz2.38*

Ajustar la variable del sistema *Path*

Si estoy trabajando en Windows, copio la ruta a donde se guardó el directorio "bin", el cual se encuentra dentro de la instalación, es posible que sea:

C:\Program Files (x86)\Graphviz2.38\bin

Y al archivo dot.exe que está dentro de bin:

C:\Program Files (x86)\Graphviz2.38\bin\dot.exe

Y agrego la primera ruta al path del *usuario* y la segunda al path de *admin* en el panel de variables de entorno.

Instalamos las librerías en Python

En el prompt de anaconda ejecutamos:

conda install python-graphviz

E instalamos pydotplus:

pip install pydotplus

Ejemplo de árbol de clasificación utilizando iris

Vamos ahora a analizar el ejemplo realizado con k-nn pero mediante la creación de un árbol de clasificación, para ello en primer lugar importamos las librerías a utilizar, en donde tenemos el clasificador de árbol de decisiones *DecisionTreeClassifier*, y la librería iris:

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import export_graphviz
import graphviz
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
```

Cargamos la librería en la variable iris.

```
iris=load_iris()
```

Utilizamos *train_test_split* para crear las variables de entrenamiento y test en base a la librería *iris*:

```
X_entrenamiento, X_test, y_entrenamiento, y_test=train_test_split(X, Y)
```

Creamos un objeto de clasificación y le indicamos la profundidad del árbol, es decir cuántos niveles bifurcaciones pretendemos. Si no indicamos un nivel de profundidad, el árbol podría generar un sobre ajuste de los datos, que si bien nos retornaría un 100% de acierto para los datos de entrenamiento, sería demasiado específico :

```
arbol=DecisionTreeClassifier(max_depth=3)
```

Y le pasamos los datos de entrenamiento:

```
arbol.fit(X_entrenamiento, y_entrenamiento)
```

En base al modelo ahora realizamos una predicción mediante score() para los datos de prueba:

```
arbol.score(X_test, y_test)
```

El modelo nos retorna un 97.3% de aciertos correctos: 0.9736842105263158

Si lo probamos con los datos de prueba:

```
arbol.score(X_entrenamiento, y_entrenamiento)
```

Obtenemos que el modelo nos retorna un 97,3% de aciertos correctos: 0.9732142857142857

Ahora podemos graficar la clasificación que el modelo ha realizado, antes podemos ver que alternativamente de agregar la librería Graphviz en las variables de entorno, podríamos lograr lo mismo de la siguiente forma:

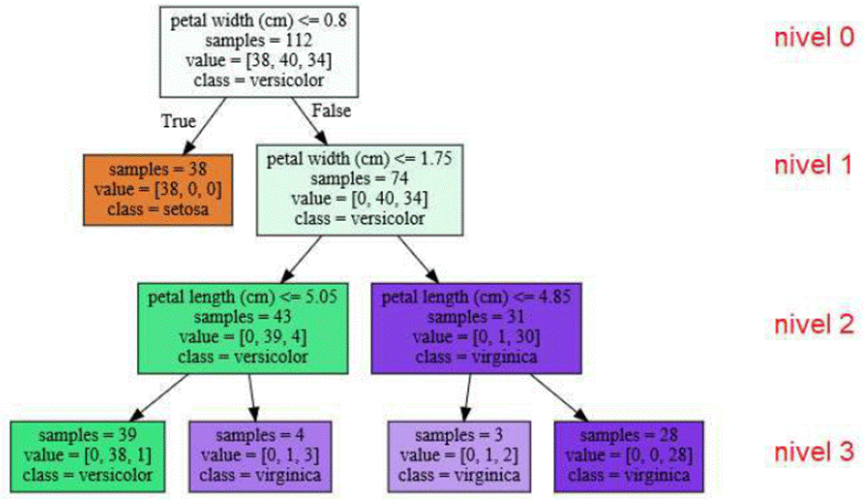
```
import os
# Usuarios de Windows:
os.environ["PATH"] += os.pathsep + 'C:\Program Files (x86)\Graphviz2.38\bin'
```

Ahora nos toca recuperar los datos que vamos a volcar dentro del esquema de árbol en un archivo mediante *export_graphviz()*

```
export_graphviz(arbol, out_file='arbol1.dot', class_names=iris.target_names,
                feature_names=iris.feature_names[:2], impurity=False, filled=True)
```

Luego recorremos el archivo y lo graficamos:

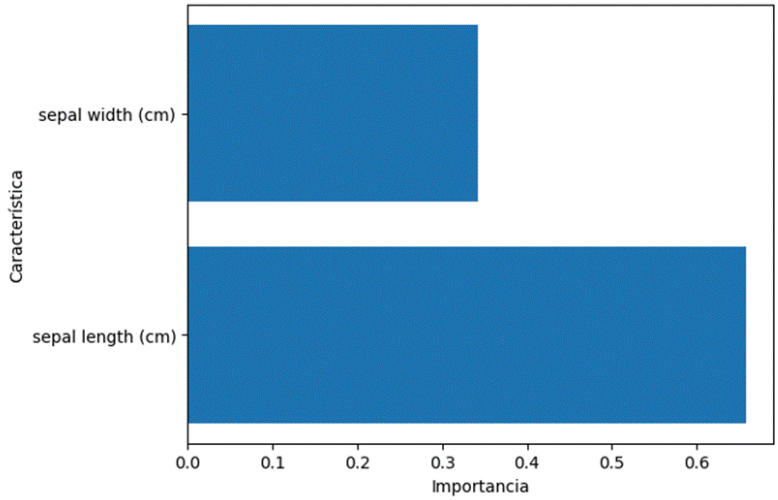
```
with open('arbol1.dot') as f:
    dot_graph=f.read()
graphviz.Source(dot_graph)
```



Dado el árbol obtenido, podemos analizar mediante *feature_imporntaces_* la importancia asignada a cada característica y ver el resultado en un gráfico, de *característica* vs *importancia asignada*:

```
caracteristica=2

plt.barh(range(caracteristica),arbol.feature_importances_)
plt.yticks(np.arange(caracteristica),iris.feature_names[:2])
plt.xlabel('Importancia')
plt.ylabel('Característica')
plt.show()
```



Del análisis sale que el modelo le ha asignado mucha importancia al ancho de los pétalos.

Otra forma de visualizar los árboles de decisión en PYTHON es usando la librería MATPLOBLIB

Importar las librerías:

from sklearn.tree import plot_tree

import matplotlib.pyplot as plt

Visualizar el árbol:

plt.figure(figsize=(10,8), dpi=150) # Ajusta el tamaño de la figura

plot_tree(modelo,feature_names=nombres_caracteristicas, class_names=nombres_clases, filled=True)

```
plt.show() # Muestra el gráfico
```

Para hacer que el árbol de decisiones sea más interpretable agregando nombres de características y clases (y configurando filled = True) podemos usar el siguiente código ejemplo

```
fn=['sepal length (cm)','sepal width (cm)','petal length (cm)','petal width (cm)']
cn=['setosa', 'versicolor', 'virginica']
fig, axes = plt.subplots(nrows = 1,ncols = 1,figsize = (4,4), dpi=300)
tree.plot_tree(clf,
               feature_names = fn,
               class_names=cn,
               filled = True);
fig.savefig('imagename.png')
```

3. Instancia de evaluación: Participación en Foro Obligatorio

Las consignas del foro de esta semana se encuentran en el mismo foro.

4. Cierre:

En el cierre de esta clase, es esencial reflexionar sobre la relevancia de los temas abordados en nuestro aprendizaje. Hemos explorado con profundidad dos algoritmos fundamentales, K-Nearest Neighbors (K-NN) y Árboles de Decisión, que se erigen como pilares en el campo del Aprendizaje Automático. Estos algoritmos, con sus enfoques únicos, nos brindan herramientas poderosas para resolver una amplia gama de problemas del mundo real. K-NN destaca en la identificación de patrones cercanos, lo que lo convierte en una elección valiosa para problemas de clasificación y regresión, como recomendaciones de productos o diagnóstico médico. Por otro lado, los Árboles de Decisión nos capacitan para tomar decisiones basadas en características clave, lo que resulta vital en casos como la detección de fraudes o la segmentación de clientes. La adquisición de este conocimiento es crucial en un mundo cada vez más impulsado por datos, ya que nos permite abordar desafíos complejos y tomar decisiones informadas respaldadas por análisis rigurosos. Estamos equipados con las herramientas necesarias para enfrentar la era del Aprendizaje Automático y aprovechar su potencial transformador en diversos campos.

Te invitamos a que nos dejes tus impresiones sobre lo que aprendimos hoy en el foro. También podés compartir links de artículos o videos que te hayan parecido interesantes mientras estudiabas este tema.

Te esperamos en los próximos encuentros. Recordá que es importante participar de los foros, de los encuentros sincrónicos y en lo posible de las tutorías presenciales.

Sitios

· "Matplotlib." *Matplotlib — Visualization with Python*, https://matplotlib.org/. Accedido el 23 Junio del 2023.

· "NumPy.org." *NumPy.org*, http://www.numpy.org/. Accedido el 23 Junio del 2023.

· "pandas." *pandas - Python Data Analysis Library*, https://pandas.pydata.org/. Accedido el 23 Junio del 2023.

· "scikit-learn." *scikit-learn: machine learning in Python — scikit-learn 1.2.2 documentation*, https://scikit-learn.org/stable/. Accedido el 23 Junio del 2023.

5. Bibliografía OBLIGATORIA

- HAN, R. (2019): *Matemáticas del Aprendizaje Automático. Introducción a la analítica de datos e Inteligencia Artificial.*
- JONES, H. (2019): *Aprendizaje Automático. El Aprendizaje Automático para principiantes que desean comprender aplicaciones, Inteligencia Artificial, Minería de Datos, Big Data y más*. Editorial Edición Kindle.
- RUSSELL, R. (2018): *Machine Learning. Guia Paso a Paso Para Implementar Algoritmos De Machine Learning Con Python*
- VIDALES, A. (2019) *Machine Learning Con Matlab*. Editorial Edición Kindle.
- CANE, A. (2019): *Programación Con Python. Guía completa para principiantes.*

- HERNÁNDEZ, V. (2019): *Aprendizaje Automático. Implementación en Inteligencia Artificial.* Editorial Edición Kindle.